

AI Agents for Everything You Do Besides Coding

Give it context — it gives you back time.

You're already using AI to write code

But a lot of a developer's time is **not about code**.

It's backlog grooming, writing user stories, setting up sprint boards, reviewing pull requests.

“

What if an AI agent could handle all of that?

”

Agenda

0. **Prerequisites** — initial setup
1. **Sprint Planning** — let the agent do the prep work
2. **Code Review** — first pass, flagged findings, published comments
3. **Personal Productivity** — morning rituals and knowledge base

0. Three Things You Need

1. **Pick your agent**

Claude Code, GitHub Copilot CLI etc.

2. **Give it access to your data**

- Azure CLI, GitHub CLI — for board, repo, pipelines
- Atlassian Remote MCP Server, Azure MCP — for Jira, Azure DevOps
- Figma MCP, Playwright MCP/CLI — optional, for design and web scraping

3. **Write your agents file — this is the highest-leverage one-time investment**

- CLAUDE.md (Claude Code) / .github/copilot-instructions.md (Copilot)
- Describe available systems, tools, project structure, tech stack
- Keep it in the repo so everyone on the team benefits

Key findings

- Find a way to give the agent access to your board, wiki, and documentation.
- Create or download skills for your tools (Azure CLI, NX etc.)
- Add a clear description of available systems and tools in agents files.
- Keep documentation in the repo.
- Update your agents file once per sprint.
- Use PAT — do not log in every 8 hours.

01 · Sprint Planning

Stop reading every ticket manually.

The agent handles prep — you handle decisions.

Sprint planning is expensive manual work

Reading every ticket. Estimating. Flagging gaps. Doing status math.

It scales badly — and it's **not where your judgment adds value.**

Sprint analysis demo

What the agent handles

- 📋 Identify user stories with **missing acceptance criteria**
- 🔗 Find related stories and suggest **implementation sequence**
- ✍️ Generate **acceptance criteria** using project docs as context
- 📊 Calculate **story points per developer** in the sprint
- ⚙️ Batch ops: change iteration path, compile sprint demo lists

02 · Code Review

Large PRs land. The agent reads everything.

You focus on what matters.

Large PRs land. You have 10 other things open

You're responsible for quality but you **can't read every line** — and you **can't afford to miss things**.

A default AI reviewer won't know your project conventions.

Two-step review workflow

1. 🔍 **Agent does first pass** → categorized findings, severity levels, suspicious spots flagged
2. 👁 **You focus on flagged spots** → your attention goes where it matters
3. 💬 **Agent posts review comments** → 2-step publish: agent prepares → you select → agent publishes

“ Build your own review command — exclude autogenerated code, skip DTO coverage checks, add bundle report analysis, enforce project-specific patterns. ”

03 · Personal Productivity

Morning rituals and a living knowledge base
that makes every session smarter.

Process daily notes

The problem: action points pile up. Unfinished notes from yesterday. You never get through everything.

The solution:

-  Unchecked tasks from yesterday → rolled into today's note
-  Meeting notes → sorted into folders with tags
-  You confirm or redirect before any file is changed

Daily notes routine demo

Your knowledge base is the agent's memory

- 🧠 Project docs alongside notes = the agent already knows your **tech stack, decisions, architecture**
- 🔄 **No re-explaining** every session
- 📝 Sprint demo outlines, architecture docs — generated with **full context**
- 📅 Context needs maintenance: **audit once per sprint**

The takeaway

“

AI doesn't replace your judgment.
It removes the **friction** between your judgment and the result.

”

Give it your context.

Let it handle the routine.

Focus on the decisions only you can make.

Q & A

Thank You

Pavel Khlebko

pavel_khlebko@epam.com

github.com/pkhlebko